CLOUD DATABASE MIGRATION DISCUSSION

Moving a Kenyan Retail-Banking MySQL Estate to the Cloud without Losing the Ledger

A critical discussion of hot physical backup, GTID replication, controlled cutover and evidence-based validation

Abstract

This discussion examines a low-downtime migration of a locally hosted MySQL 8.0 database that supports a Kenyan retail-banking application into a cloud-hosted MySQL-compatible service. The argument is deliberately unromantic. Minimal downtime is not produced by copying faster. It is produced by separating two jobs that are routinely collapsed into one: the movement of bulk state, and the capture of change that continues while that state is in transit. We therefore treat Percona XtraBackup as a way of taking a transactionally consistent physical photograph of the source while the bank remains open, and Global Transaction Identifier (GTID) binary-log replication as the mechanism that keeps the photograph current until the moment of authority transfer.

The rehearsal used synthetic customer, account, beneficiary and transaction records denominated in Kenya shillings, across branch, ATM, mobile, agent and card channels. No production extract was touched. That choice is not a convenience; it is a control. A migration rehearsal that cannot be run at realistic cardinality without exposing personal data is not a rehearsal a Kenyan bank can defend.

The central claim is that technical equality of two databases is necessary and still insufficient. Wei and Chen's checksum principle is used here not as a slogan but as a warning: incomplete comparison methods create a false sense of completion. In a ledger, two tables can have identical row counts and still disagree on balances, posting status or referential links. The assurance model therefore triangulates schema comparison, referential-integrity tests, financial aggregates at DECIMAL precision, chunked checksums, GTID position, application tests and an audit trail that an independent reviewer could reconstruct. The same section is honest about residual risk. Asynchronous replication can reproduce logical corruption as faithfully as it reproduces legitimate postings. Managed-service privilege models strip objects that on-premises operators take for granted. And once the cloud accepts the first write, simple reversal is no longer a rollback; it is a second migration.

1. Introduction and critical position

A database migration in banking is not a data-copy exercise dressed up in a project plan. It is a controlled transfer of system-of-record responsibility from one failure domain to another. The technical objective is to preserve atomicity, consistency, isolation and durability. The organisational objective is to keep branches, agents and mobile channels usable, to remain

accountable to the Central Bank of Kenya, and to leave behind evidence that would survive a post-incident review. Those two objectives pull against each other. Aggressive parallelism shortens elapsed time and contends for the same disks the production workload needs. Long validation increases assurance and enlarges the change window. Immediate rollback looks prudent on a slide and becomes a dual-write problem the moment the first cloud transaction commits.

We adopted a staged replication design because it makes that trade-off explicit rather than hiding it behind a vendor claim of “zero downtime”. The full dataset is copied while the local database remains online. Subsequent transactions are replayed continuously from the binary log. Only the final change of authority requires a write pause. Amazon Web Services documents the same general logic for Aurora MySQL: a physical backup accelerates bulk movement, and binary-log replication then synchronises the transactions that occurred after the backup was taken (Amazon Web Services, 2026a). The design therefore converts hours of outage into a brief, managed cutover. It does not, and should not, promise literal zero downtime. Domain-name or secret rotation, connection-pool drainage, final consistency checks and application restart still impose a finite interruption. Claiming otherwise would be the kind of overselling that later has to be explained to a board.

The regulatory setting is not an appendix. The Central Bank of Kenya’s Guidance Note on Cybersecurity frames business continuity as the timely maintenance or recovery of operations, and it identifies databases, privileged access and third-party connections as cyber-risk concerns (Central Bank of Kenya, 2017). That framing converts operational choices — replication encryption, role separation, key custody, rollback rehearsal — into governance requirements. The Data Protection Act, 2019 adds a second constraint: even synthetic rehearsals sit next to a production estate that will eventually carry personal data, and cross-border processing of that estate is not a purely architectural decision. Success, in this setting, is not “the replica is running”. Success is demonstrated technical equality, preserved business invariants, and named owners who have accepted the residual risk.

This write-up is written in the first-person plural because that is how the work actually happened. “We” is a small migration cell — database engineering, application support, information security, and a business owner from retail operations — working through a rehearsal that was designed to fail safely. Where the original draft treated the migration as a finished method, this revision treats it as an argument that has to survive contact with lag, privilege models and human approval gates.

2. Scenario, scope and assumptions

The institution in view is a Kenyan tier-one-style retail bank moving a locally hosted application database to a managed MySQL-compatible service in a region the bank has already approved. The target could be Amazon Aurora MySQL, Amazon RDS for MySQL, Azure Database for MySQL or Google Cloud SQL, provided the selected service supports the source major version, a binary-log replication path, encryption, logging and private networking. To keep the demonstration reproducible, the worked commands assume

MySQL 8.0 on both sides and a provisioned cloud endpoint reachable through an IPsec VPN or a dedicated private circuit.

We were careful not to pretend this is a core-banking system migration. Core ledgers bring posting engines, end-of-day batches, interest accrual and a political economy of vendor lock-in that this paper does not try to settle. What we migrated is the kind of channel-adjacent store that Kenyan retail banks actually run beside the core: customers, accounts, beneficiaries, a transaction ledger and channel references. That is already enough to expose the hard problems — DECIMAL money, idempotent posting, orphan prevention, and the social fact that mobile and agent channels do not pause because a database team has a maintenance window.

Table 1. Illustrative environment and the reason each choice is not arbitrary

Component	Illustrative specification	Critical rationale
Source	MySQL 8.0 on Linux; InnoDB; ~500 GB; local data centre; UTC	InnoDB is the only engine we would trust for a crash-consistent physical backup. UTC removes the Nairobi/UTC ambiguity that otherwise appears in value-time reconciliations.
Target	Managed MySQL-compatible HA; Multi-AZ; encrypted storage; private endpoint	Managed resilience reduces night-work. It also removes SUPER-class privileges that on-premises runbooks quietly assume.
Workload	Customers, accounts, beneficiaries, txn ledger, channel refs	Enough cardinality and money semantics to test assurance without cloning a CBS.
Availability objective	Bulk copy online; short write freeze at cutover	Downtime is compressed by pre-synchronisation, not by unsafe dual writes.
Security boundary	TLS 1.2+; VPN/private link; least privilege; KMS/HSM keys	Transit and rest protection with custody the bank can still explain.
Recovery point	Full backup plus retained binary logs	Replay to a known transaction boundary; rollback evidence, not folklore.

2.1 Synthetic banking data model

Identifiers are fictitious, names are generated, account numbers are tokenised, and no production extract is used. The point of the model is to reproduce scale, cardinality, data types and transaction patterns without increasing privacy exposure. That is the only rehearsal posture we were prepared to defend to information security.

```
CREATE DATABASE bank_app CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
CREATE TABLE customer (
  customer_id BIGINT PRIMARY KEY,
  full_name VARCHAR(120) NOT NULL,
  national_id_token CHAR(64) NOT NULL UNIQUE,
  risk_rating ENUM('LOW','MEDIUM','HIGH'),
  created_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6)
) ENGINE=InnoDB;
CREATE TABLE account (
  account_id BIGINT PRIMARY KEY, customer_id BIGINT NOT NULL,
  account_no_token CHAR(64) NOT NULL UNIQUE,
  currency CHAR(3) NOT NULL DEFAULT 'KES',
  status ENUM('ACTIVE','DORMANT','BLOCKED','CLOSED') NOT NULL,
  book_balance DECIMAL(18,2) NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
) ENGINE=InnoDB;
CREATE TABLE txn_ledger (
  txn_id BIGINT PRIMARY KEY, account_id BIGINT NOT NULL,
  value_time TIMESTAMP(6) NOT NULL,
  channel ENUM('BRANCH','ATM','MOBILE','AGENT','CARD'),
  direction ENUM('DR','CR') NOT NULL,
  amount DECIMAL(18,2) NOT NULL, currency CHAR(3) NOT NULL,
```

```
  idempotency_key CHAR(36) NOT NULL UNIQUE,
  posting_status ENUM('POSTED','REVERSED','PENDING'),
  FOREIGN KEY (account_id) REFERENCES account(account_id)
) ENGINE=InnoDB;
```

Two modelling decisions carry more weight than they appear to. First, money is DECIMAL(18,2), never floating point. A migration that “almost” reconciles because somebody hashed a DOUBLE is not a banking migration. Second, every posting carries an idempotency key. Cutover is the moment when retries, double-submits and channel timeouts become most likely; without idempotency the smoke test itself can create the inconsistency it is supposed to detect.

3. How downtime is actually reduced

This is the question the assignment asks most directly, and it is the place where vendor language is least trustworthy. The naive mental model of migration is: stop the application, copy the database, start the application somewhere else. Elapsed outage then equals copy time plus restore time plus courage. For a 500 GB InnoDB estate that is an evening, not a window.

The design we used inverts that model. Outage is no longer a function of data volume. It becomes a function of three much smaller quantities: the time needed to drain writers, the time needed for the replica to apply the last few binary-log events, and the time needed to switch connection secrets and prove that the new authority is safe. Everything else happens while customers are still being served from the source.

3.1 Two clocks, not one

It helps to think of the migration as two clocks running at once. Clock A is the bulk transfer: XtraBackup copies pages and redo, the backup is prepared, encrypted, moved across a private path, and restored into the cloud. Clock A is long — in rehearsal, hours rather than minutes — but it does not stop the source. Clock B is change capture: every committed transaction after the backup’s consistent point is written to the binary log and later applied on the target. Clock B runs for as long as Clock A needs, and then a little longer.

Cutover is the instant at which we stop Clock A’s source from producing new writes, wait until Clock B reads zero remaining work, and then point the application at the target. AWS states the same separation for Aurora: physical migration is faster than logical reload for large databases and does not impose the source-performance penalty of a blocking dump; binary-log replication then closes the gap created after the backup (Amazon Web Services, 2026a, 2026b). Percona’s own replica-provisioning guidance is built on the identical idea — seed from a hot backup, then start replication from the recorded coordinates (Percona, 2026a, 2026b).

3.2 Why GTID matters more than file names

Traditional binary-log positioning asks an operator to copy a filename and an offset out of a backup metadata file and type them into CHANGE REPLICATION SOURCE TO without transposition error. GTID-based replication identifies each committed transaction independently of the file that happens to contain it (Oracle, 2026a). Auto-positioning

therefore removes a class of operator error that is otherwise concentrated at 02:00. MySQL's own documentation is explicit that required binary logs must not be purged before the recipient catches up (Oracle, 2026b). That single sentence is why we sized binlog retention to the whole rehearsal-plus-contingency period rather than to ordinary operational housekeeping. A "successful" backup that sits behind a purge is not a seed; it is a dead end.

There is a step the first draft under-specified and which Percona's GTID replica procedure treats as mandatory on a self-managed restore: after copy-back, the replica's gtid_purged must be set from the GTID set recorded in xtrabackup_binlog_info before START REPLICA (Percona, 2026b). Managed import paths often apply that coordinate for you. Self-managed paths do not. Missing the step produces a replica that looks healthy and is silently missing history.

3.3 What "real-time" does and does not mean

We used the phrase "real-time replication" in conversation with the business because it is the language they already use for disaster-recovery copies. Technically, what we ran is asynchronous, row-based, GTID replication. Asynchronous is a design choice, not a defect. Synchronous commit across a Nairobi-to-cloud WAN would have imported WAN jitter into every customer posting — the opposite of the availability objective. The price of asynchrony is lag. Lag is why we never treated Seconds_Behind_Source as a green light on its own. During errors that value can be NULL or flattering. The go/no-go view combined receiver and applier state, GTID-set comparison, relay-log growth, apply throughput and the error log.

Note: Incomplete or inaccurate comparison methods leave a migration looking finished when the target is not yet a trustworthy copy of the source. (Wei and Chen, 2014)

That distinction mattered in the room. Operations instinctively wanted to "just switch". Risk wanted to "just keep checking". The replication-until-convergence model gave both sides a shared artefact: a lag graph that had to flatten, and a GTID set that had to be a subset, before anyone touched a connection string.

3.4 Why we refused dual-write

The fashionable alternative to a write freeze is to open both databases to the application and reconcile later. In a document store with commutative updates that can be made to work. In a debit/credit ledger it is a split-brain invitation. Without a consensus protocol and an idempotent business design that the existing application does not have, two writers create two truths. Manne's survey of high-availability banking databases is useful here precisely because it refuses to treat replication topology as a fashion: synchronous replication buys zero data loss at a latency cost; asynchronous replication buys source performance at an RPO cost (Manne, 2024). Our cutover is a short, explicit move from the second model to a single new authority, not an attempt to occupy both.

Stripe's public account of zero-downtime data movement is instructive as a contrast, not a template. Their constraint — keep the critical phase shorter than a planned primary failover, inside the application's retry budget — is the right way to think about a cutover (Stripe, 2024).

We do not have Stripe's document-database fabric. We do have retry budgets on mobile and agent channels. The write freeze was therefore sized to sit inside those budgets, not to satisfy a slogan.

Table 2. Where time is spent, and which of it is visible to customers

Activity	Typical rehearsal duration	Customer-visible?
Discovery, compatibility, approvals, rehearsal planning	Days to weeks	No
Hot XtraBackup + prepare + encrypt + SHA-256 manifest	1–4 hours at 500 GB	No (source stays online)
Private transfer and cloud restore / provider import	1–6 hours, path-dependent	No
GTID catch-up under live traffic	Hours to a day, lag-dependent	No
Layered validation while replica is live	Hours, overlapping catch-up	No
Write freeze, final apply, reconcile, switch, smoke test	Minutes (target: inside channel retry budget)	Yes — this is the outage

4. Tool selection and architecture

Tool choice is where migration papers become catalogues. We tried to make each selection do argumentative work.

Percona XtraBackup 8.0 is the baseline tool because it takes an online physical backup of InnoDB and then prepares the copy by applying transaction logs, producing a consistent recovery image without a prolonged lock (Percona, 2026c). For a large InnoDB database that is generally a better first move than mysqldump, which serialises schema and row reconstruction and pays for that simplicity at restore time. AWS Prescriptive Guidance makes the same size argument: above roughly 100 GB, physical copy restores materially faster than logical reload (Amazon Web Services, 2026c). Physical portability is narrower, however. XtraBackup 8.0 is bound to the MySQL 8.0 page, redo and data-dictionary formats; it will not reach back to 5.7, and minor-version skew on the restore side is a documented failure mode (Percona, 2026c). A logical dump remains the right tool for small estates, selective schemas or cross-version transformation. Its simplicity should not be confused with superior recovery time.

Native asynchronous MySQL replication is the change-data-capture layer. We preferred it to a third-party CDC pipeline for a pedestrian reason: the bank already operates binary logs, the target already speaks MySQL replication, and every extra product is another privilege model, another key, and another vendor conversation with CBK-facing outsourcing questions. Row-based logging records changed row images rather than re-executing statements that may be non-deterministic. That matters for a ledger with triggers and for any future use of NOW(), UUID() or user-defined functions in posting paths.

Percona Toolkit's pt-table-checksum is the content-comparison tool. It checksums deterministic chunks on the source and lets those checksum statements replicate, so differences appear as DIFFS rather than as a hope that COUNT(*) was enough (Percona, 2026d). Wei and Chen (2014) supply the theoretical warrant. Their paper exists because migration verification in practice is often incomplete, inaccurate, or both; checksums are

proposed as a feasible way to verify correctness without a full pairwise walk of every byte. In a bank that insight has to be extended. Equal hashes on a chunk do not prove that a posting_status of PENDING was the business state anyone intended. Checksums catch drift. They do not replace financial reconciliation.

Table 3. Control layers and why each exists

Layer	Chosen control	Why it is there
Baseline	XtraBackup 8.0 full hot backup	Fast physical copy; prepared, crash-consistent image; limited source blocking.
Change capture	GTID + row-based binlog	Replays committed changes after the baseline; deterministic positioning.
Transfer	xbstream/compression + TLS over VPN	Volume reduction; interception resistance across a trust boundary.
File integrity	SHA-256 manifest	Detects transport or staging alteration. Does not prove logical equality.
Logical integrity	pt-table-checksum + SQL reconciliations	Separates “the file arrived” from “the ledger still adds up”.
Observability	performance_schema, cloud metrics, SIEM	Lag, errors, privilege use and odd access have to be visible, not inferred.
Rollback	Retained source + immutable backup + tested reverse plan	Avoids irreversible cutover without licensing dual-write.

Google Cloud SQL’s physical-file migration path is worth naming because it confirms the pattern is no longer an AWS peculiarity: XtraBackup files loaded to object storage, replica attached, catch-up by replication, claimed order-of-magnitude improvement over logical dump for compatible 5.7/8.0 sources (Google Cloud, 2026). The organisational implication is that the method is portable. The engineering implication is that each provider’s unsupported-object list is different, so a provider pre-check is not bureaucracy. It is the moment we discover that routines, dynamic privileges or plugins will not come across.

5. End-to-end procedure

5.1 Discovery, readiness and governance

We started with an inventory that made people impatient and later saved us. Schemas, tables, sizes, engines, character sets, collations, views, triggers, events, routines, definers, users and privileges. Non-InnoDB tables were treated as blockers for the physical path, not as footnotes. Peak transaction rate and long-running queries were measured because the replication channel has to beat the source write rate with headroom; otherwise lag grows for the entire rehearsal and the “short freeze” becomes a long one.

Compatibility is where cloud marketing and on-premises habit collide. SQL mode, lower_case_table_names, time zone, max_allowed_packet, reserved words, and the provider’s list of unsupported plugins and privileges all had to be checked before anyone ran a backup. Aurora and RDS physical import, to take the best-documented case, will not import functions, procedures, views, events and triggers defined by the reserved rdsadmin user; they strip unsupported privileges from imported accounts; Aurora MySQL 3 does not import dynamic privileges; plugins do not come across; and user tables in the mysql schema are not migrated (Amazon Web Services, 2026b). A runbook that assumes “restore equals the source” is therefore already wrong.

- Approve the change, the rollback decision points, RTO/RPO, reconciliation thresholds, processing location, encryption-key ownership, and named technical and business signatories.
- Run at least one full-scale rehearsal with production-like synthetic data. A paper runbook does not reveal throughput bottlenecks, hidden definers, or the variance in restore duration that later wrecks a Saturday window.
- Treat the rehearsal evidence pack as part of the design, not as paperwork to be assembled after go-live.

5.2 Configure the source

Durability settings were kept strict during migration. Weakening sync_binlog or innodb_flush_log_at_trx_commit to “go faster” would have undermined the consistency the exercise exists to protect. Retention was sized from analysis, not from a copied blog post.

```
[mysqld]
server_id=101
log_bin=mysql-bin
binlog_format=ROW
gtid_mode=ON
enforce_gtid_consistency=ON
binlog_row_image=FULL
binlog_expire_logs_seconds=1209600   # 14 days; size from migration analysis
sync_binlog=1
innodb_flush_log_at_trx_commit=1
```

Accounts were minted from a vault, required SSL, and were written so they could be revoked without ceremony.

```
CREATE USER 'migration_repl'@'10.%' IDENTIFIED BY '<vault-secret>' REQUIRE SSL;
GRANT REPLICATION SLAVE, REPLICATION CLIENT ON *.* TO 'migration_repl'@'10.%';
CREATE USER 'xtrabackup'@'localhost' IDENTIFIED BY '<local-vault-secret>';
GRANT BACKUP_ADMIN, PROCESS, RELOAD, LOCK TABLES, REPLICATION CLIENT ON *.* TO 'xtrabackup'@'localhost';
```

5.3 Encrypted hot physical backup

The pattern we would take to a Kenyan bank’s change board is an online physical full backup, encrypted under a bank-controlled key, accompanied by transaction logs for point-in-time recovery, written to hardened staging and copied to a second protected location before anyone calls it a seed. That is a common enterprise pattern. It is not a claim that every Kenyan bank uses this exact product. Control value depends on independent restore testing, separate key custody, write-protected copies and reconciliation evidence. “The command exited zero” is evidence of execution, not of recoverability.

```
xtrabackup --backup --login-path=xbk \
  --target-dir=/backup/bank_full_20260925 \
  --parallel=4 --compress --compress-threads=4
xtrabackup --prepare --target-dir=/backup/bank_full_20260925
find /backup/bank_full_20260925 -type f -print0 | sort -z | \
  xargs -0 sha256sum > /backup/bank_full_20260925.SHA256
```

Where catch-up volume threatened to dominate the freeze, we planned an incremental XtraBackup after the full, following the AWS pattern of shrinking the binary-log debt the target must apply (Amazon Web Services, 2026d). That is a practical refinement, not a change of architecture.

5.4 Secure transfer

```
rsync -aH --numeric-ids --partial --append-verify \
  -e 'ssh -o StrictHostKeyChecking=yes' \
  /backup/bank_full_20260925/ migration-gateway:/cloud-staging/bank_full_20260925/
sha256sum -c bank_full_20260925.SHA256
```

The path used private connectivity, host verification, restricted security groups, rate controls and central logging. SHA-256 answers a narrow question: did these bytes change in transit? It does not answer whether the restored ledger is the source ledger. Treating the manifest as a completion certificate is one of the ways migrations acquire false confidence.

5.5 Restore and establish GTID replication

If the cloud service permits physical import, the prepared backup is restored through the provider's mechanism — for Aurora, from XtraBackup files in S3 for compatible versions (Amazon Web Services, 2026b). Those constraints are why the pre-check exists. Where physical import is unsupported, the fallback is a provider migration service or a logical load followed by binlog replication. Falling back is not failure; insisting on a physical path the platform will not honour is.

On a self-managed target the sequence is stop, copy-back, fix ownership, start, apply the GTID set from backup metadata, then point replication at the source with auto-positioning and TLS.

```
systemctl stop mysqld
xtrabackup --copy-back --target-dir=/cloud-staging/bank_full_20260925
chown -R mysql:mysql /var/lib/mysql
systemctl start mysqld
-- set gtid_purged from xtrabackup_binlog_info BEFORE starting replication
CHANGE REPLICATION SOURCE TO
  SOURCE_HOST='onprem-db.internal', SOURCE_PORT=3306,
  SOURCE_USER='migration_repl', SOURCE_PASSWORD='<secret-manager>',
  SOURCE_SSL=1, SOURCE_AUTO_POSITION=1;
START REPLICA;
```

Any skipped transaction required root-cause analysis and formal approval. Silently setting sql_replica_skip_counter would have broken the audit trail we claimed to be building.

5.6 Validation before cutover

Wei and Chen (2014) argue that verification must go beyond incomplete comparison and that checksums earn their keep because they make a correctness claim that row counts cannot. In a bank that argument has to be widened. Two databases can have equal counts and still differ in balances, posting status, referential links or decimal precision. We therefore layered cheap tests for breadth under expensive tests for financial materiality.

Table 4. Validation layers, illustrative checks and the acceptance line we were willing to sign

Layer	Illustrative check	Acceptance criterion
Structural	Schema diff; collation; indexes; partitions; routines; definers	No unexplained difference; exceptions risk-accepted and tested.
Completeness	COUNT(*) by table and date partition	Source equals target at the agreed GTID / cutover boundary.
Financial aggregates	SUM of DR/CR and closing balances by currency and date	Exact DECIMAL equality; no float comparison.
Integrity	Orphans; foreign keys; duplicate business keys; NULL profile	Zero new violations; baseline defects separately documented.

Layer	Illustrative check	Acceptance criterion
Content	Chunked deterministic checksums on critical tables	All checked chunks equal; mismatches investigated and rechecked.
Replication	GTID_SUBSET plus receiver / applier health	Target has executed the source cutover GTID set; no errors.
Application	Login, balance, posting, reversal, statement, idempotency	Expected outputs, audit events, latency within limits.
Security	TLS, encryption, secrets, least privilege, audit, vuln scan	Controls effective; evidence in the change record.

The SQL we actually ran at the agreed boundary was unglamorous on purpose.

```
SELECT currency, posting_status, COUNT(*) AS rows_n,
  SUM(CASE WHEN direction='DR' THEN amount ELSE 0 END) AS debit_total,
  SUM(CASE WHEN direction='CR' THEN amount ELSE 0 END) AS credit_total
FROM txn_ledger GROUP BY currency, posting_status
ORDER BY currency, posting_status;
SELECT COUNT(*) AS orphan_transactions
FROM txn_ledger t LEFT JOIN account a ON a.account_id=t.account_id
WHERE a.account_id IS NULL;
SELECT COUNT(*) AS duplicate_idempotency_keys
FROM (SELECT idempotency_key FROM txn_ledger
      GROUP BY idempotency_key HAVING COUNT(*)>1) d;
```

For very large tables, pt-table-checksum compared chunks while replication stayed active. We tested it first for load and privileges. Hashing used stable column order, explicit NULL representation and deterministic key ordering. Otherwise equal data produces different hashes, or different data looks equal through sloppy concatenation. Checksums are persuasive evidence. They are not a mathematical proof against every collision or every semantic error.

5.7 Controlled cutover

Table 5. Cutover clock

Point	Action
T−60 min	Confirm target health, lag trend, backup integrity, rollback readiness, named approvers.
T−10 min	Drain non-essential jobs; pause batch posting; lower connection TTL where the application allows.
T−2 min	Maintenance / read-only mode. Stop writers. Verify no unauthorised sessions remain.
T0	Record source GTID set and final financial-control totals. Let the target apply remaining events.
T+1	Verify GTID convergence, zero replication errors, final critical reconciliations.
T+2	Switch secret / endpoint to the cloud target; recycle connection pools.
T+3	Smoke tests and controlled low-value synthetic posting; check audit logs and monitors.
T+5	Open service gradually; watch error rate and latency; take technical and business sign-off.

Downtime shrinks because the 500 GB baseline and almost all subsequent changes are already in the cloud. The unavailable period covers write quiescence, final catch-up, validation and connection switching. That is the whole trick, and it only works if nobody starts writing to both sides.

5.8 Rollback and the point of no return

Rollback is easy before the first cloud write. If a critical check fails, the application stays on the local source and replication is stopped under change control. After cloud writes begin, DNS reversal is not rollback; the on-premises database is stale. The bank then has to establish tested reverse replication, replay an audited transaction queue, or restore the prior platform from the cloud's authoritative boundary. The go/no-go immediately before enabling cloud writes is therefore the irreversible threshold. We said that out loud in the CAB, twice.

- Keep the source read-only and access-restricted for the approved stabilisation period. No heroic "just this one update".
- Rotate or revoke temporary migration and replication accounts; destroy staging files; keep immutable evidence.
- Run enhanced monitoring and daily reconciliations through hypercare; finish a post-implementation review.
- Update CMDB, DR runbooks, data-flow diagrams and support ownership before anyone calls the job done.
- Decommission only after retention, legal, audit, business and security approvals exist in writing.

6. Challenges, and what they do to the argument

A master's discussion that only narrates a happy path is a brochure. The rehearsal produced a short list of difficulties that actually change the design, not just the risk register.

Table 6. Challenges that survived contact with the rehearsal

Challenge	Why it is difficult	Control, and what still remains
Replication lag	WAN latency and peak writes can outrun apply capacity and stretch the freeze.	Parallel appliers, sizing, QoS, rehearsal. Asynchronous replication still permits lag.
Logical corruption	Replication copies fraudulent deletes as faithfully as it copies legitimate credits.	Immutable backups, delayed replica / PITR, anomaly detection. Replication is not a backup.
Provider incompatibility	Managed services restrict SUPER, plugins, definers and system schemas.	Automated pre-checks. Some features need redesign, not migration.
Long transactions and DDL	Large transactions delay apply; DDL may block or differ on the target.	Freeze schema change; find long transactions; schedule around the business.

Challenge	Why it is difficult	Control, and what still remains
Financial precision	Type or collation change alters decimals, identifiers or order.	Exact DECIMAL, schema diff, aggregates and row checks. Semantics need a business reader.
Security and privacy	Backup copies enlarge the attack surface and cross organisational lines.	Encryption, private net, least privilege, DLP, key separation, timed destruction. Insider risk remains.
False confidence in counts	Equal counts do not prove equal values or relationships.	Checksums, constraints, business reconciliation. Sampling cannot prove completeness.
Rollback after first write	The source stops being authoritative the moment the cloud accepts a posting.	Named authority boundary and reverse path. Rollback may itself need downtime.

6.1 Minimal downtime is the wrong primary metric

The most important conceptual error in this kind of programme is to treat elapsed outage as the dominant success measure. A migration that completes in seconds and cannot demonstrate ledger equivalence is a control failure. The opposite error is also real: insisting on an exhaustive byte-by-byte comparison of every historical object during the freeze recreates the outage the design was meant to avoid. The stronger posture is risk-tiered assurance. Exhaustive reconciliation for current balances and financially material ledgers. Chunked checksums for high-risk tables. Structural verification for all objects. Statistically justified sampling only for lower-risk archival content. That is an explicit bargain between assurance cost and operational resilience, and it should be written into the CAB paper rather than invented at T−10 minutes.

6.2 Shared responsibility does not move accountability

Managed high availability can improve resilience. It does not move the bank's accountability for schema design, access control, key management, application behaviour, data-residency decisions or provider exit. CBK's 2017 note is blunt, in the way regulators are blunt, that boards and senior management own cyber risk and that third-party connections are part of that risk, not a way of exporting it (Central Bank of Kenya, 2017). Equating a provider's certification with the bank's compliance is the category error the Guidance Note exists to prevent. Standard Chartered's later public discussion of Aurora for core-adjacent workloads is useful as an existence proof that large banks do run MySQL-family engines in the cloud, and equally useful as a reminder that they do so with a stated RPO/RTO and a pilot-light DR pattern rather than with faith in a logo (Loup and Craig, 2024).

6.3 Privacy is not solved by synthetic data

Synthetic data protected the rehearsal. It does not protect the production cutover that the rehearsal is meant to enable. The Data Protection Act, 2019 still governs the personal data that will eventually sit on the target, including the conditions under which that data may be processed outside Kenya. The migration architecture therefore has to carry a documented processing location, a lawful-basis and transfer assessment, and a destruction certificate for

staging copies. Those artefacts belong in the evidence pack beside the SHA-256 manifest. Leaving them to “Legal will handle it” is how technical teams discover, after the replica is healthy, that the region they chose cannot be used.

7. Evidence pack and what “done” meant

An independent reviewer should be able to reconstruct what moved, by whom, at which transaction boundary, under which controls, with what result. If they cannot, we do not have a migration record. We have a story.

- Approved architecture, data-flow diagram, impact assessment, change record, separation-of-duties matrix.
- Source and target inventories, version and parameter comparison, provider compatibility report, exception register.
- XtraBackup logs, SHA-256 manifest, restore-test evidence, object-storage audit logs, retained binlog range.
- Replication snapshots, GTID sets, lag metrics, error logs, proof of encrypted transport.
- Signed structural, row-count, checksum, financial-reconciliation and application-test results.
- Cutover timestamps, endpoint and secret changes, smoke-test output, monitoring, customer-impact note.
- Go/no-go and rollback decisions, business sign-off, security sign-off, post-implementation review.

Success meant: no unexplained schema or data variance; the target had executed the agreed source GTID set; debit, credit and balance totals matched exactly; application and audit functions behaved; approved downtime and error thresholds were met; temporary access was revoked; recoverability was demonstrated by tested backup and rollback evidence. “Replica running” did not pass.

8. Conclusion

The method separates bulk movement from live change capture. Percona XtraBackup produces a recoverable physical baseline without stopping the source for the length of the copy. GTID-based binary-log replication then applies the transactions that happen while that baseline is in transit. A short write freeze creates a single, auditable authority boundary for final catch-up, reconciliation and endpoint switch. Downtime falls because almost all of the work has already been done in the background, not because anyone found a way to copy 500 GB instantaneously.

The critical finding is that tool choice is secondary to control design. Physical backup offers speed and imports compatibility constraints. Replication reduces outage and will carry corruption if corruption is what the source emits. Checksums raise confidence and still need deterministic construction plus a business reading of money. Cloud high availability does

not transfer accountability away from the bank. Reliability is the combination of tested recovery, transaction-boundary evidence, layered validation, formal governance and a rollback plan that remains honest after the first cloud write.

In a Kenyan banking context that combination is the point. Continuity, confidentiality, integrity and public trust are not slogans to print on a title page. They are the tests the rehearsal was built to fail against, on purpose, before anyone pointed a production application at a new failure domain.

References

Amazon Web Services (2026a) Migrating data to an Amazon Aurora MySQL DB cluster. Amazon Aurora User Guide. Available at: https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraMySQL.Migrating.html (Accessed: 25 September 2026).

Amazon Web Services (2026b) Physical migration from MySQL by using Percona XtraBackup and Amazon S3. Amazon Aurora User Guide. Available at: https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraMySQL.Migrating.ExtMySQL.S3.html (Accessed: 25 September 2026).

Amazon Web Services (2026c) Percona XtraBackup. AWS Prescriptive Guidance: Migrating large MySQL and MariaDB databases to the AWS Cloud. Available at: https://docs.aws.amazon.com/prescriptive-guidance/latest/migration-large-mysql-mariadb-databases/percona-xtrabackup.html (Accessed: 25 September 2026).

Amazon Web Services (2026d) Migrate on-premises MySQL databases to Aurora MySQL using Percona XtraBackup, Amazon EFS, and Amazon S3. AWS Prescriptive Guidance Patterns. Available at: https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/migrate-on-premises-mysql-databases-to-aurora-mysql-using-percona-xtrabackup-amazon-efs-and-amazon-s3.html (Accessed: 25 September 2026).

Central Bank of Kenya (2017) Guidance Note on Cybersecurity for the Banking Sector. Nairobi: Central Bank of Kenya. Available at: https://www.centralbank.go.ke/2017/08/18/cbk-guidance-note-cybersecurity/ (Accessed: 25 September 2026).

Google Cloud (2026) Migrate to Cloud SQL from an XtraBackup physical file. Cloud SQL for MySQL documentation. Available at: https://docs.cloud.google.com/sql/docs/mysql/migrate-xtrabackup-physical-file (Accessed: 25 September 2026).

Loup, X. and Craig, J. (2024) Amazon Aurora for core banking systems. AWS for Industries. Available at: https://aws.amazon.com/blogs/industries/amazon-aurora-for-core-banking-systems/ (Accessed: 25 September 2026).

Manne, U.K. (2024) ‘High-availability databases in global banking: ensuring continuous operations and data integrity’, International Journal of Scientific Research in Computer Science, Engineering and Information Technology, 10(5), pp. 410–ff. Available at: https://doi.org/10.32628/CSEIT24105109 (Accessed: 25 September 2026).

Oracle (2026a) Setting up replication using GTIDs. MySQL 8.0 Reference Manual, §19.1.3.4. Available at: https://dev.mysql.com/doc/refman/8.0/en/replication-gtids-howto.html (Accessed: 25 September 2026).

Oracle (2026b) Cloning for replication. MySQL 8.0 Reference Manual, §7.6.7.7. Available at: https://dev.mysql.com/doc/refman/8.0/en/clone-plugin-replication.html (Accessed: 25 September 2026).

Percona (2026a) How to set up a replica for replication with Percona XtraBackup 8.0. Available at: https://docs.percona.com/percona-xtrabackup/8.0/set-up-replication.html (Accessed: 25 September 2026).

Percona (2026b) How to create a new (or repair a broken) GTID-based replica. Available at: https://docs.percona.com/percona-xtrabackup/8.0/create-gtid-replica.html (Accessed: 25 September 2026).

Percona (2026c) About Percona XtraBackup 8.0. Available at: https://docs.percona.com/percona-xtrabackup/8.0/about-xtrabackup.html (Accessed: 25 September 2026).

Percona (2026d) pt-table-checksum. Percona Toolkit documentation. Available at: https://docs.percona.com/percona-toolkit/pt-table-checksum.html (Accessed: 25 September 2026).

Republic of Kenya (2019) Data Protection Act, No. 24 of 2019. Nairobi: Government Printer.

Stripe (2024) How Stripe's document databases supported 99.999% uptime with zero-downtime data migrations. Stripe Engineering Blog. Available at: https://stripe.com/blog/how-stripes-document-databases-supported-99.999-uptime-with-zero-downtime-data-migrations (Accessed: 25 September 2026).

Wei, B. and Chen, T.X. (2014) Verifying data migration correctness: The checksum principle. RTI Press. Available at: https://doi.org/10.3768/rtipress.2014.op.0019.1403 (Accessed: 25 September 2026).

Appendix A. Concise migration runbook

Phase	Key activities	Primary evidence
Plan	Inventory, compatibility, classification, approvals, rehearsal, rollback design	Approved plan and exception register
Prepare	Enable GTID and binlogs; provision private target; accounts; TLS; monitoring	Configuration and access records
Seed	Hot backup, prepare, encrypt, manifest, transfer, restore	Backup / restore logs and hashes
Replicate	Start GTID replication; monitor lag and errors; validate continuously	GTID and performance_schema output
Cut over	Quiesce writes; final catch-up; reconcile; switch endpoint; smoke test	Signed go/no-go and cutover log
Stabilise	Hypercare; daily reconciliation; revoke access; update DR and CMDB	Monitoring and closure report
Decommission	Retain read-only; obtain approvals; securely erase and attest	Deletion certificate and audit sign-off